

[프로젝트 협업 대화 기록: 교내 도서관 자리 예매 프로그램 V1.0]

 **나의 1차 요청:** 나는 파이썬 프로젝트로 *[교내 도서관 자리 예매 프로그램]**을 만들기로 결정했어. 이번 1차 과제에서는 조건문(if)이나 리스트, 반복문(while)을 쓰지 않고, 오직 '변수 선언'과 '입출력(input, print)만 사용해야 해. 이 제한 조건 안에서 영화관 예매 시스템의 첫 시작 화면과 데이터를 입력받고 결과를 보여주는 화면(V1.0)을 어떻게 구성하면 좋을지 필요한 변수 목록(자료형 포함)과 출력 시나리오를 설계해 줘.

 **Gemini의 1차 제안:** 제한된 조건(변수, 입출력만 사용) 내에서 프로그램을 설계하는 것은 데이터의 흐름을 이해하는 아주 좋은 연습입니다. 조건문이 없기 때문에 '선택'이나 '예외 처리'는 불가능하지만, 사용자에게 정보를 입력받고 이를 정리해서 보여주는 '영수증'이나 '확인서' 형태의 구성을 제안합니다. (변수 목록 설계: student_name, student_id, library_floor, seat_number, start_time, use_hours) (3단계 출력 시나리오 및 파이썬 V1.0 기초 코드 제공)

제한된 조건(**변수, 입출력만 사용**) 내에서 프로그램을 설계하는 것은 데이터의 흐름을 이해하는 아주 좋은 연습입니다. 조건문이 없기 때문에 '선택'이나 '예외 처리'는 불가능하지만, 사용자에게 정보를 입력받고 이를 정리해서 보여주는 '**영수증**'이나 '**확인서**' 형태의 구성을 제안합니다.

1. 데이터 설계 (변수 목록)

사용자로부터 어떤 정보를 받을지 정의합니다. 조건문이 없으므로 모든 변수는 한 번씩만 입력받고 그대로 출력하는 구조로 설계했습니다.

변수명	자료형 (Type)	용도
student_name	String (문자열)	학생 이름 입력
student_id	String (문자열)	학번 입력 (계산이 필요 없으므로 문자열 권장)
library_floor	String (문자열)	희망 층수 (예: 1층, 2층)
seat_number	String (문자열)	좌석 번호
start_time	String (문자열)	이용 시작 시간 (예: 14:00)

use_hours	Int (정수)	이용 시간 (수치 데이터)
-----------	----------	----------------

2. 출력 시나리오 설계

화면은 크게 ****[메인 타이틀] → [정보 입력] → [예매 결과 출력]****의 3단계로 구성됩니다.

단계 1: 시스템 시작 화면

프로그램의 목적을 알리는 환영 문구를 출력합니다.

출력 예시:

Plaintext

[OO대학교 도서관 좌석 예매 시스템 V1.0]
안녕하세요! 좌석 예약을 위해 정보를 입력해 주세요.

단계 2: 데이터 입력 (Input)

사용자로부터 필요한 정보를 순차적으로 받습니다.

출력 예시:

Plaintext

성함: (사용자 입력)
학번: (사용자 입력)
희망 좌석 번호: (사용자 입력)
이용 예정 시간(시간 단위): (사용자 입력)

단계 3: 예매 결과 확인서 (Output)

입력받은 변수들을 활용해 최종 예약 내역을 시각적으로 깔끔하게 보여줍니다.

출력 예시:

[예약이 완료되었습니다]
신청자: 홍길동 (20241234)
예약 좌석: 2층 45번 좌석
이용 시간: 14:00부터 3시간 동안

※ 본 화면을 캡처하여 좌석 이용 시 지참해 주세요.

3. 파이썬 코드 예시 (V1.0)

작성하신 시나리오를 코드로 구현하면 다음과 같습니다.

Python

```
# 1. 시작 화면 출력
print("=====")
print("  OO대학교 도서관 좌석 예매 시스템 V1.0  ")
print("=====")
print()

# 2. 정보 입력 (변수 선언 및 입력)
student_name = input("학생 이름을 입력하세요: ")
student_id = input("학번을 입력하세요: ")
library_floor = input("이용하실 층수를 입력하세요: ")
seat_number = input("좌석 번호를 입력하세요: ")
start_time = input("이용 시작 시간을 입력하세요 (예: 13:00): ")
use_hours = input("이용하실 시간(숫자만)을 입력하세요: ")

# 3. 결과 출력 (입력받은 변수 활용)
print()
print("-----")
print("      [좌석 예약 확인서]      ")
print("-----")
```

```
print(f"▶ 신청자 성명: {student_name}")
print(f"▶ 신청자 학번: {student_id}")
print(f"▶ 예약된 좌석: {library_floor} {seat_number}")
print(f"▶ 이용 예정: {start_time}부터 {use_hours}시간 동안")
print("-----")
print(" 이용 10분 전까지 입실해 주시기 바랍니다. ")
print("=====")
```

💡 팁

- **f-string 활용:** print(f"이름: {student_name}") 형식을 사용하면 변수와 문자열을 아주 깔끔하게 조합할 수 있어 V1.0 과제에서 좋은 점수를 받을 수 있을 거예요.
- **시각화:** print() 문 안에 -나 =를 활용해 경계선을 만들어주면 '시스템' 같은 느낌이 훨씬 강해집니다.

이 설계를 바탕으로 과제를 진행해 보시는 건 어떨까요? 혹시 출력 양식을 더 화려하게 만들고 싶거나 궁금한 점이 있다면 언제든지 물어봐 주세요!

👤 **나의 2차 요청:** 더 자세히는 사용자가 본인의 정보를 입력 후 자신의 입실 시간과 퇴실 예정 시간을 입력해 몇 분이 남았는지 중간중간 확인이 가능하면 좋겠어. 예약 확인서는 되게 좋은 거 같아.

🤖 **Gemini의 2차 제안:** 좋은 아이디어예요! 하지만 조건문(if)과 반복문(while)을 쓸 수 없다는 제약이 있으므로, 실행 시점에 한 번 계산해서 보여주는 방식으로 설계를 보완해 볼게요. (시간 계산을 위한 start_hour, start_min 등 변수 추가) (조건문 없이 산술 연산자 +, -, *, //, % 만을 사용하여 퇴실 시간 및 잔여 시간을 계산하는 파이썬 V1.1 코드 설계 제공)

좋은 아이디어예요! 예약 확인서에 **퇴실 시간**과 **잔여 시간**까지 포함하면 훨씬 친절한 프로그램이 되겠네요.

하지만 이번 과제에서 '**조건문(if)**'과 '**반복문(while)**'을 쓸 수 없다는 점이 아주 까다로운 제약사항이에요. 보통 '중간중간 확인'하려면 반복문이 필요하지만, 1차 과제 규칙에 맞춰 ****[실행 시점에 딱 한 번 계산해서 보여주는 방식]****으로 설계를 보완해 볼게요.

시간 계산을 위해 '분(Minute)' 단위로 변환하여 처리하는 로직을 추가했습니다.

1. 보완된 데이터 설계 (변수 추가)

기존 변수에 시간 계산을 위한 정수형(Int) 변수들을 추가합니다.

변수명	자료형	용도
start_hour	Int	입실 시간(시)
start_min	Int	입실 시간(분)
use_total_min	Int	총 이용할 시간(분 단위 입력)
current_hour	Int	현재 시간(시) - 남은 시간 확인용
current_min	Int	현재 시간(분) - 남은 시간 확인용

2. 출력 시나리오 (잔여 시간 계산 포함)

조건문 없이 **산술 연산자(+, -, *, /, //, %)**만 사용하여 입실/퇴실 시간과 잔여 시간을 계산합니다.

[입력 단계]

- 사용자 기본 정보 입력 (이름, 학번 등)
- 입실 시각 입력 (예: 14시 30분)
- 이용 예정 시간 입력 (예: 120분)
- 현재 시각 입력 (예: 15시 10분)

[계산 로직 (내부 처리)]

- 퇴실 예정 시각:
 - 전체 분 = (입실 시 * 60) + 입실 분 + 이용 분
 - 퇴실 시 = 전체 분 // 60
 - 퇴실 분 = 전체 분 % 60
- 남은 시간 확인:
 - 입실 총분 = (입실 시 * 60) + 입실 분
 - 현재 총분 = (현재 시 * 60) + 현재 분
 - 잔여 분 = (입실 총분 + 이용 분) - 현재 총분

3. 파이썬 코드 설계 (V1.1)

Python

1. 정보 입력

```
print("--- [도서관 좌석 예약 및 시간 계산기] ---")
```

```
name = input("이름: ")
```

```
s_hour = int(input("입실 시(24시 형태): "))
```

```
s_min = int(input("입실 분: "))
```

```
duration = int(input("이용 예정 시간(분 단위): "))
```

2. 퇴실 시간 계산 (조건문 없이 산술 연산만 사용)

```
total_start_min = (s_hour * 60) + s_min
```

```
total_end_min = total_start_min + duration
```

```
end_hour = (total_end_min // 60) % 24 # 24시간이 넘어가면 0시로 리셋
```

```
end_min = total_end_min % 60
```

3. 현재 시각 입력 (중간 확인용)

```
print("\n--- [현재 상태 확인] ---")
```

```
curr_hour = int(input("현재 시(24시 형태): "))
```

```
curr_min = int(input("현재 분: "))
```

4. 잔여 시간 계산

```
total_curr_min = (curr_hour * 60) + curr_min
```

```
remaining_min = total_end_min - total_curr_min
```

5. 최종 결과 출력

```
print("\n" + "="*40)
```

```
print(f" {name} 님의 이용 현황")
```

```
print("-" * 40)
```

```
print(f"▶ 입실 시간: {s_hour:02d}:{s_min:02d}")
```

```
print(f"▶ 퇴실 예정: {end_hour:02d}:{end_min:02d}")
```

```
print(f"▶ 현재까지 남은 시간: {remaining_min}분")
```

```
print("="*40)
```

```
print("※ 이용 시간이 종료되면 퇴실 버튼을 눌러주세요.")
```

💡 과제 제출용 포인트 (Wit & Tip)

1. **:02d 포매팅**: 14:5라고 나오는 대신 14:05처럼 두 자리로 예쁘게 출력되게 하는 마법입니다. 교수님이 "오, 기본기 이상의 센스가 있네?"라고 생각하실 부분이에요.
2. **데이터 타입 변환**: input()으로 받은 건 문자열이라 계산이 안 돼요. int()로 감싸서 숫자로 바꾸는 과정을 설계안에 포함했다는 점을 강조하세요.
3. **한계점 명시**: "현재 버전(V1.1)은 조건문을 사용하지 않으므로, 현재 시간이 퇴실 시간보다 늦은 경우 마이너스(-) 분이 출력될 수 있습니다. 이는 V2.0(조건문 도입)에서 해결할 예정입니다."라고 주석을 달면 아주 완벽한 과제가 됩니다.

이렇게 입실/퇴실 시간까지 계산해 주는 기능, 과제 요구사항에 딱 맞으면서도 실용적이지 않나요?

👤 **나의 3차 요청**: 내가 원하는 시나리오는 사용자가 도서관에 가서 자신의 이름과 학번을 입력한 후 입실 시간과 퇴실 시간을 입력해. 그다음으로 몇 층의 열람실 몇 번의 자리를 이용할지 선택하면 예약 확인증으로 "몇 층 몇 번 좌석이 예매되었습니다"라는 문구가 나오고 자신의 잔여 시간이 궁금할 때에 중간중간 프로그램에 가서 확인할 수 있는 프로그램을 만들고 싶은 거야.

🤖 **Gemini의 3차 제안**: 사용자의 의도를 완벽히 이해했습니다! 제한 조건을 지키면서도 '키오스크'나 '앱'을 사용하는 것 같은 흐름으로 시나리오를 다시 짜보겠습니다. 조건문이 없으므로 [예약 단계]와 [조회 단계]가 한 번의 실행 흐름으로 이어지는 방식이 됩니다. (사용자 흐름에 맞춘 변수 재정립 및 예약/조회 원스톱 V1.2 코드 시나리오 제공)

사용자의 의도를 완벽히 이해했습니다! 학교 과제의 ****제한 조건(if, while, list 사용 금지)****을 지키면서도, 사용자가 마치 '키오스크'나 '앱'을 사용하는 것 같은 흐름으로 시나리오를 다시 짜보겠습니다.

조건문을 쓸 수 없기 때문에, 이 프로그램은 ****[예약 단계]****와 ****[조회 단계]****가 한 번의 실행 흐름 (Linear Flow)으로 쭉 이어지는 방식이 됩니다.

1. 프로그램 설계 개요 (V1.2)

- **핵심 로직**: 모든 시간 데이터를 '분(Minute)' 단위로 환산하여 계산합니다.
- **사용자 흐름**:
 1. 개인정보 및 예약 정보 입력 (층, 좌석, 입/퇴실 시간)
 2. 예약 완료 확인증 출력
 3. (프로그램이 종료되지 않았다고 가정하고) 바로 현재 시각을 입력하여 잔여 시간 확인

2. 데이터 설계 (변수 목록)

변수 분류	변수명 (자료형)	설명
개인 정보	user_name (Str), user_id (Str)	이름, 학번
좌석 정보	lib_floor (Str), seat_num (Str)	층수, 좌석 번호
예약 시간	in_h, in_m (Int) / out_h, out_m (Int)	입실 시/분, 퇴실 시/분
계산 변수	total_in, total_out (Int)	입/퇴실 시간을 '분'으로 환산
조회 시점	now_h, now_m (Int)	현재 시각 (잔여 시간 확인용)

3. 출력 시나리오 및 실행 흐름

[STEP 1: 예약 정보 입력]

사용자에게 예약에 필요한 모든 정보를 한 번에 물어봅니다.

출력: "환영합니다! 예약을 위해 이름, 학번, 이용 층수(예: 3층), 좌석 번호를 차례대로 입력해 주세요."

입력: (각 정보를 변수에 저장)

[STEP 2: 이용 시간 설정]

입실 시간과 퇴실 시간을 숫자로 입력받습니다.

출력: "입실 시간(시, 분)과 퇴실 예정 시간(시, 분)을 입력해 주세요."

계산: > * total_in = (in_h * 60) + in_m

- total_out = (out_h * 60) + out_m

[STEP 3: 예약 확인증 발급]

입력받은 정보를 조합하여 화려한 확인증을 출력합니다.

출력:

Plaintext

[예약 확인증]

이용자: 홍길동 (20240001)

예약 좌석: 3층 12번 좌석

이용 시간: 14:00 ~ 18:00

"3층 12번 좌석이 성공적으로 예매되었습니다."

[STEP 4: 실시간 잔여 시간 확인]

프로그램 하단에 현재 시각을 입력하면 남은 시간을 바로 계산해 주는 코너를 만듭니다.

출력: "현재 시각을 입력하면 남은 이용 시간을 알려드립니다."

입력: now_h, now_m

계산: $\text{remain_min} = \text{total_out} - ((\text{now_h} * 60) + \text{now_m})$

결과 출력: "홍길동님, 퇴실까지 남은 시간은 **[{remain_min}]**분입니다."

4. 파이썬 코드 구성 예시

Python

```
# --- 1. 정보 입력 단계 ---
```

```
print("=== [교내 도서관 좌석 예매 시스템] ===")
```

```
user_name = input("성함: ")
```

```
user_id = input("학번: ")
```

```
lib_floor = input("이용 층수: ")
```

```
seat_num = input("좌석 번호: ")
```

```
print("\n--- [시간 설정 (24시 형식)] ---")
```

```
in_h = int(input("입실 시: "))
```

```
in_m = int(input("입실 분: "))
```

```
out_h = int(input("퇴실 시: "))
```

```
out_m = int(input("퇴실 분: "))
```

```
# --- 2. 예약 확인증 출력 ---
```

```
print("\n" + "="*30)
```

```
print(f"      예 약 확 인 증")
print("-" * 30)
print(f" {lib_floor} {seat_num} 좌석이 예매되었습니다.")
print(f" 이용자: {user_name}({user_id})")
print(f" 이용 시간: {in_h:02d}:{in_m:02d} ~ {out_h:02d}:{out_m:02d}")
print("="*30)
```

```
# --- 3. 중간 시간 확인 단계 ---
```

```
print("\n[ 실시간 잔여 시간 조회 ]")
```

```
now_h = int(input("현재 시: "))
```

```
now_m = int(input("현재 분: "))
```

```
# 계산 로직
```

```
total_out = (out_h * 60) + out_m
```

```
total_now = (now_h * 60) + now_m
```

```
remain_min = total_out - total_now
```

```
print(f"\n{user_name}님, 퇴실까지 {remain_min}분 남았습니다.")
```

```
print("열공하세요!")
```

💡 이 시나리오의 장점 (과제 포인트)

1. **현실성**: 입실/퇴실 시간을 각각 입력받아 '이용 시간대'를 명시함으로써 실제 예매 시스템과 유사합니다.
2. **수학적 해결**: 조건문(if) 없이도 **산술 연산**만으로 퇴실 시각과 남은 시간을 정확히 계산해내는 논리력을 보여줄 수 있습니다.
3. **사용자 경험(UX)**: 확인증 문구("몇층 몇번 좌석이 예매되었습니다")를 명확히 포함하여 요구사항을 충실히 반영했습니다.

이 설계대로라면 1차 과제에서 '변수와 입출력 활용 능력' 부문에서 아주 좋은 점수를 받으실 수 있을 거예요! 추가로 더 꾸미고 싶은 부분이 있다면 말씀해 주세요.

👤 **나의 4차 요청**: 예상 기능과 메뉴를 직관적으로 (최소 5개) 1. 2. 3. 4. 이런 형태로 정리해줄 수 있을까?

🤖 **Gemini의 4차 제안**: 제한된 조건 내에서도 프로그램이 풍성해 보이도록, 사용자 여정(User Journey)에 맞춘 5가지 핵심 기능과 메뉴를 정리해 드릴게요.

1. [STEP 1] 이용자 본인 확인 (이름/학번 입력)
2. [STEP 2] 학습 공간 선택 (층수 및 좌석 번호 지정)
3. [STEP 3] 이용 시간 설정 (입실 및 퇴실 시각 입력)
4. [STEP 4] 이용 현황 중간 점검 (현재 시각 기반 잔여 시간 확인)
5. [STEP 5] 최종 예매 확인증 및 안내 문구 출력

제한된 조건(변수, 입출력만 사용) 내에서도 프로그램이 풍성해 보일 수 있도록, **사용자 여정(User Journey)**에 맞춘 5가지 핵심 기능과 메뉴를 정리해 드릴게요.

조건문을 쓸 수 없으므로, 메뉴는 '선택'하는 방식이 아니라 **프로그램이 실행되면서 순차적으로** 거쳐가는 '단계'로 구성됩니다.

도서관 좌석 예매 시스템 V1.0 예상 기능 & 메뉴

1. 사용자 인증 및 식별 (User Check-in)

기능: 이용자의 고유 정보(이름, 학번)를 입력받아 데이터로 저장합니다.

직관적 메뉴: [STEP 1] 이용자 본인 확인 (이름/학번 입력)

2. 좌석 위치 지정 (Space Selection)

기능: 도서관 내의 구체적인 위치(층수, 좌석 번호)를 텍스트로 입력받아 확정합니다.

직관적 메뉴: [STEP 2] 학습 공간 선택 (층수 및 좌석 번호 지정)

3. 이용 시간 설정 및 퇴실 계산 (Time Scheduling)

기능: 입실 시간과 퇴실 예정 시간을 입력받아, 총 이용 가능 시간을 계산합니다.

직관적 메뉴: [STEP 3] 이용 시간 설정 (입실 및 퇴실 시각 입력)

4. 실시간 잔여 시간 조회 (Live Remaining Time)

기능: 사용자가 조회하고 싶은 '현재 시각'을 입력하면, 퇴실 시각과의 차이를 계산해 남은 분 (minute)을 보여줍니다.

직관적 메뉴: [STEP 4] 이용 현황 중간 점검 (현재 시각 기반 잔여 시간 확인)

5. 디지털 예약 확인증 발급 (Digital Receipt)

기능: 앞서 입력한 모든 정보(이름, 좌석, 시간, 잔여 시간)를 종합하여 하나의 완성된 텍스트 양식으로 출력합니다.

직관적 메뉴: [STEP 5] 최종 예매 확인증 및 안내 문구 출력

💡 구성 팁

조건문을 사용하지 못하는 1차 과제 특성상, 메뉴를 나열할 때 ****"사용자가 이 순서대로 정보를 입력하며 예약을 완성해 나간다"***는 시나리오를 강조하는 것이 기획 의도를 설명하기에 훨씬 유리합니다.

이 기능들이 다 포함된다면, 단순한 입출력 프로그램 이상의 **'논리적인 서비스'** 느낌이 물씬 풍길 거예요! 더 구체적으로 다듬고 싶은 메뉴가 있을까요?

👤 나의 5차 요청: 해당 프로그램에 필요한 (변수 5개 자료형 3개)와 조건문과 반복문이 없이 입실/퇴실 시간 차이를 계산하는 로직을 구상해줘.

🤖 Gemini의 5차 제안: 조건문과 반복문 없이 오직 산술 연산만으로 시간을 계산하는 로직입니다.

- 필수 변수 및 자료형: user_info(String), seat_info(String), start_h/start_m(Integer), end_h/end_m(Integer), usage_rate(Float)
- 시간 차이 계산 로직: 모든 시간을 '분(Minute)'으로 통일하여 계산.
 - 전체 분 = (시 * 60) + 분
 - 총 이용 분 = 퇴실 전체 분 - 입실 전체 분
 - 시 = 총 이용 분 // 60, 분 = 총 이용 분 % 60

조건문과 반복문 없이 오직 **산술 연산**만으로 시간을 계산하는 로직은 이번 과제의 핵심이자 가장 똑똑해 보일 수 있는 부분입니다. 요청하신 변수와 자료형, 그리고 핵심 계산 로직을 정리해 드릴게요.

1. 프로그램 필수 변수 및 자료형 (5개 변수 / 3개 자료형)

조건문이 없으므로, 입력을 받을 때부터 계산이 가능한 자료형으로 변환하는 것이 포인트입니다.

변수명	자료형 (Type)	의미 및 용도
user_info	String (문자열)	사용자 이름과 학번 (출력용)
seat_info	String (문자열)	선택한 층수와 좌석 번호 (출력용)
start_h, start_m	Integer (정수)	입실 시각(시/분) - 산술 연산용

end_h, end_m	Integer (정수)	퇴실 예정 시각(시/분) - 산술 연산용
usage_rate	Float (실수)	(선택 사항) 시간당 이용률 등 소수점이 필요한 지표

2. 시간 차이 계산 로직 (수학적 설계)

조건문(if) 없이 시간을 계산할 때 가장 큰 문제는 **시(Hour)와 분(Minute)의 단위가 다르다**는 것입니다. 이를 해결하는 가장 깔끔한 방법은 **모든 시간을 '분'으로 통일**하는 것입니다.

[핵심 공식]

- 1. 시간을 분으로 환산: $\$전체\ 분 = (시\ \times 60) + 분\$$
- 2. 시간 차이(이용 시간) 계산: $\$총\ 이용\ 분 = 퇴실\ 전체\ 분 - 입실\ 전체\ 분\$$
- 3. 다시 시/분으로 분리 (출력용):
 - 시간: $\$총\ 이용\ 분 // 60\$$ (몫 연산자)
 - 분: $\$총\ 이용\ 분 \% 60\$$ (나머지 연산자)

3. 코드 적용 시나리오 (Logic Flow)

파이썬의 산술 연산자만 사용하여 아래와 같이 구성할 수 있습니다.

Python

```
# 1. 데이터 입력 (문자열 및 정수형 변환)
user_info = input("이름과 학번을 입력하세요: ")
seat_info = input("예약할 층과 좌석을 입력하세요: ")

in_h = int(input("입실 시: "))
in_m = int(input("입실 분: "))
out_h = int(input("퇴실 시: "))
out_m = int(input("퇴실 분: "))

# 2. 산술 연산을 이용한 시간 차이 계산
# 모든 시각을 '분' 단위로 변환하여 차이를 구함
total_in_minutes = (in_h * 60) + in_m
```

```
total_out_minutes = (out_h * 60) + out_m
duration_minutes = total_out_minutes - total_in_minutes
```

3. 결과 출력 (나머지와 몫 연산 활용)

```
print(f"\n[{user_info}]님, {seat_info} 예매 완료!")
print(f"총 이용 예정 시간은 {duration_minutes // 60}시간 {duration_minutes % 60}분입니다.")
```

💡 알고리즘 포인트

- **몫 연산자(//):** 소수점을 버리고 '시간' 단위만 추출할 때 사용합니다.
- **나머지 연산자(%):** 60으로 나눈 나머지를 통해 '분' 단위만 추출할 때 사용합니다.
- **제약 사항 언급:** "본 로직은 당일 이용을 원칙으로 하며, 자정이 넘는 예약은 V2.0(조건문 도입)에서 업데이트 예정"이라고 주석을 달아두면 완벽합니다.

이렇게 설계하면 조건문 없이도 사용자가 입력한 시간에 따른 정확한 이용 시간을 계산해줄 수 있습니다. 이 로직을 과제에 그대로 녹여내면 아주 논리적인 결과물이 될 거예요!

👤 **나의 6차 요청:** 방금까지 우리가 나눈 대화를 요약해주고 나는 지금 너랑 해당 프로젝트를 협업한다는 느낌으로 수행 중인데 너가 봤을 때에 부족한 점으로는 무엇이 있을까?

🤖 **Gemini의 6차 제안:** 지금까지 [교내 도서관 좌석 예매 프로그램 V1.0]의 뼈대를 탄탄하게 잡았습니다. (대화 및 설계 내용 요약 제공) 협업 파트너로서 본 '부족한 점'과 보완 가이드:

1. 예외 상황에 대한 '안내' 부족: 조건문이 없으므로 입력창 옆에 (주의: 24시간 형식의 숫자만 입력) 같은 가이드 문구 배치 필요.
2. 시각적 구분(UI)의 단순함: `print("-" * 30)` 등을 활용해 입력 구역과 결과 출력 구역 분리.
3. 데이터의 휘발성: V2.0에서 반복문(while)을 도입해 데이터를 유지하며 반복 조회할 수 있도록 개선 예정이라는 향후 발전 방향(Roadmap)을 보고서에 명시할 것을 권장.

성공적인 과제 제출과 프로젝트 마무리를 응원합니다!